# Asolaria System Card for OpenAI Board Review

Date: 2026-03-09

Authoring basis: local source tree, local reports, live runbooks, connector code, bridge runtime docs

Sensitivity: sanitized narrative, no secrets or token values

## 1. Board Thesis

Asolaria is what happens when a frontier model stops being treated like a chat window and becomes a governed operating substrate.

It is not a single agent and it is not a wrapper around one model. It is a local-first control plane that combines:

- frontier-model reasoning lanes
- governed tool execution
- desktop and phone actuation
- relay-backed multi-agent coordination
- graph-native runtime telemetry
- explicit approval and authority transitions
- persistent memory, task, project, and mistake stores

The reason this should matter to OpenAI is simple: Asolaria already looks like the kind of environment advanced operators actually want. It treats Codex, Symphony, remote MCP, desktop control, mobile control, NotebookLM, GCP, and collaboration systems as parts of one auditable operational fabric.

## 2. What Asolaria Actually Is

At a systems level, Asolaria is a local control plane with six distinct layers:

- Interaction plane: web UI, app shell, mobile console, approval surfaces, phone-safe operator views
- Execution plane: async job queue, skill runner, connector dispatch, worker router, local CLI execution
- Integration plane: Google, GCP, Vertex, NotebookLM, GitHub, Slack, Telegram, Microsoft, Atlassian, Symphony, Augment, external-provider MCP
- Realtime media plane: voice, transcription, caption bridge, meeting-assist pathways
- Governance plane: authority modes, approvals, allowlists, audit verification, readiness drills, relay policy
- Ops plane: health checks, supervisors, watchdogs, tunnel keepers, startup scripts, recovery runbooks, white-glove sweeps

This is reinforced in the local architecture and startup docs:

- Canonical source is explicitly separated from runtime snapshots.

- Startup has a role model for `Asolaria Supervisor`, `Vertex`, `Rook`, and `Falcon`.
- Authority transitions are two-step and rollbackable.
- Tool execution is governed through gateway policy instead of being treated as a blind side effect.

# 3. Why This Is More Than "An App"

Most "AI agent systems" stop at prompt routing.

Asolaria goes several steps further:

- It has a real worker topology.
- It has a governed execution gateway.
- It records graph events and action manifests.
- It has a mistake ledger and mistake taxonomy.
- It has a project store and persistent memory store.
- It has mobile and desktop control surfaces.
- It has a relay stack that lets multiple Codex instances cooperate hands-free across machines.
- It has explicit public-release, handover, and bootstrap documentation.

That combination is the differentiator. The interesting artifact here is not the UI. The interesting artifact is the operational spine.

# 4. Runtime Snapshot and Proven Scale of Surface Area

Recent local runtime reports establish a concrete footprint:

- Express API routes: 331 total
- Connector modules in `src/connectors`: 40
- Indexed skills: 53
- Indexed tool actions: 32
- Indexed active mistake patterns: 17 in the 2026-03-05 snapshot
- Observed colony count in the 2026-03-08 runtime report: 6 total, 6 online, 3 active
- Queue state in the same runtime pass: clear at time of scan

The important point is not just the count. It is that the surfaces are already split into distinct operational domains instead of being blended into one opaque agent runtime.

# 5. Core Execution Model

## 5.1 Brain orchestration

The orchestrator already supports a multi-provider chain rather than a single hardcoded model path. The local provider order is built around:

- `codex`
- `anthropic`
- `gemini-api`
- `vertex`
- `gemini-cli`
- `openclaw`
- `cursor`
- `antigravity`

That is the right control-plane posture: keep the orchestration local, keep the model layer pluggable, and keep fallback logic policy-aware.

### 5.2 Worker routing

The worker router formalizes which work should go where instead of improvising every task. Current worker lanes are:

- `local_codex`
- `abacus`
- `symphony`

That means Asolaria can already distinguish:

- highest-trust local engineering work
- bounded external research or packet work
- long-running workflow execution tied to backlog systems

### 5.3 Async execution

Job execution is queue-backed, concurrency-limited, and stateful:

- queued
- running
- completed
- failed

That may sound basic, but it matters. A credible control plane needs a scheduler, not just a prompt handler.

## 6. Governance, Safety, and Authority

This is one of the strongest parts of the system.

### 6.1 Authority modes

Gateway authority modes are explicit:

- `operator_primary`
- `shared_control`
- `asolaria_primary`

Transitions are guarded by:

- a request step
- a confirmation step
- readiness evaluation
- rollback to `operator_primary`

### 6.2 Tool authority

Tool invocation is policy-gated, approval-aware, and audited. High-risk tools are not simply callable because a model asked for them.

Important characteristics:

- tool invocations can require one-time approval
- approval records are bound to tool id plus input hash
- approval records are stored and expire by TTL
- audit integrity can hard-lock control writes on verification failure

This is the difference between "agent with tools" and "operational runtime with tool governance."

### 6.3 Audit integrity

Asolaria records:

- authority events
- approval events
- tool mode changes
- graph runtime events
- action manifests

The audit design is explicitly tamper-aware, and the graph runtime has its own event and manifest streams.

## 7. Connectors and External Systems

The connector layer is already broad enough to show what a serious frontier-model control plane looks like.

### 7.1 Full connector inventory

Current connector modules:

- `abacusConnector`

- anthropicCliConnector
- atlassianConnector
- augmentContextConnector
- avatarNpcConnector
- capturesConnector
- chromeConnector
- codexConnector
- cursorAgentConnector
- desktopCaptureConnector
- desktopControlConnector
- desktopDisplayConnector
- externalBrainConnector
- gcpConnector
- geminiApiConnector
- geminiCliConnector
- geminiLiveConnector
- githubConnector
- googleConnector
- googleWorkspaceCliConnector
- localOpsConnector
- localRebuildConnector
- microsoftConnector
- notebooklmConnector
- openclawConnector
- padConnector
- phoneBrowserHistoryConnector
- phoneDeveloperOptionsConnector
- phoneMirrorConnector
- phoneProMirrorConnector
- screenshotConnector
- slackConnector
- symphonyConnector
- systemPaths
- telegramConnector

- `uiVisualAuditConnector`
- `vertexConnector`
- `voiceConnector`
- `wolConnector`
- `workerRouter`

## 7.2 What this means in practice

This is not just "integration breadth." It means Asolaria can operate across:

- reasoning lanes
- cloud APIs
- enterprise notebooks
- ticketing and collaboration systems
- browser and screen surfaces
- desktop windows and input
- phone devices and ADB-backed controls
- wake, capture, display, and network-adjacent operational scripts

## 7.3 Symphony

Symphony is integrated as a managed sidecar, not as a loose external dependency.

Important attributes:

- local repo-aware configuration
- workflow file binding
- Linear project slug binding
- guarded local start, stop, restart
- Windows-aware startup path for the reference runtime
- worker-router support for long-running issue execution

This is especially important for OpenAI because it shows Asolaria treating `openai/symphony` as a real operational lane rather than a demo.

## 7.4 Linear

Linear is not just present as a generic integration. It appears in the system as part of the Symphony execution path and orchestration semantics:

- issue pickup
- project slug routing
- long-running work handoff

- Playwright-based session support

## 7.5 Auggie / Augment Context

Augment is integrated as a read-only context sidecar.

This is a strong design choice.

Instead of turning Augment into a second uncontrolled "brain," Asolaria uses it as:

- local CLI retrieval for the same workspace
- remote MCP read-only context bridge
- compact tool surface restricted to `codebase-retrieval`
- token-reduction-aware context path for external providers

The bridge path is narrow, stateless, and bearer-protected. That is the right shape.

## 7.6 GCP and Vertex

GCP and Vertex are first-class, not afterthoughts.

The GCP integration includes:

- service-account-backed configuration
- allowed project enforcement
- read-only default scopes
- allowlisted admin service enabling

The Vertex integration includes:

- project, location, and model configuration
- daily budget configuration
- request/token ceilings
- warning thresholds
- budget usage tracking

This is exactly the kind of operational discipline that makes a control plane credible.

## 7.7 Google / Workspace / NotebookLM

The Google layer spans:

- Google auth and account linkage
- Google Workspace CLI
- NotebookLM integration
- NotebookLM enterprise readiness and notebook listing

- Gmail and Calendar surfaces in the mobile console

One internal report records NotebookLM enterprise readiness as live and synced against the project `winged-complex-390417`.

### 7.8 OpenAI-native media path

The voice layer is also relevant to OpenAI directly.

The local connector defaults its cloud transcription model to `gpt-4o-mini-transcribe` and can fall back across other configured media paths, including Gemini live audio turns. That means Asolaria is not only using frontier models for reasoning. It is already treating OpenAI as part of a multimodal operating path.

### 7.9 Collaboration systems

Collaboration surfaces already include:

- GitHub
- Slack
- Telegram
- Microsoft
- Atlassian

This means Asolaria can sit where actual teams work, not only where models work.

## 8. Desktop, Browser, and Phone Abilities

This is one of the most impressive parts of the system because it crosses abstraction levels cleanly.

### 8.1 Desktop abilities

The desktop layer includes:

- window list
- active window introspection
- window focus
- minimize, maximize, restore, close
- pointer movement
- click and double-click
- scroll
- type
- key input
- capture

This is not pseudo-computer-use. It is a script-backed desktop input host with explicit action validation.

## 8.2 Browser abilities

Browser and Chrome-related surfaces include:

- open URL
- inspect profile state
- screenshot and capture
- browser task routing
- UI visual audit

This gives Asolaria both human-visible and non-visual browser pathways.

## 8.3 Phone abilities

The phone stack is already unusually mature.

It includes:

- phone mirror capture
- pro mirror support
- browser history inspection
- developer options inspection and control
- USB and device selection logic
- WhatsApp-targeted voice capture pathways
- mobile console surfaces for approvals, status, and chat
- Termux-aware phone-side command execution lanes

The runtime and runbooks also show:

- phone tunnel monitor
- phone background keeper
- phone route selection in startup
- explicit fallback ordering across USB, Wi-Fi or VPN or local network, then tunnel or cellular fallback

This is not a toy mobile companion. It is an actual phone control plane.

## 8.4 AnyDesk and remote operator assistance

Asolaria's broader operating environment also includes AnyDesk-assisted workflows:

- AnyDesk screenshot capture
- OCR loops
- remote prompt detection
- focus management

- guarded nudges and operator notifications

That matters because it shows the system is being used to control real-world remote assistance paths, not only local demo surfaces.

# 9. Graph Runtime and the GNN Story

This section needs to be precise.

## 9.1 What is live now

The graph runtime is real and active today.

Live pieces include:

- graph event log
- action manifest log
- graph runtime APIs
- graph runtime UI
- edge-risk scoring at event build time
- training dataset export for graph-runtime windows

The graph runtime records typed events such as:

- actor
- subject
- target
- context
- policy
- detail
- risk

This is exactly the right substrate for graph-native reasoning over an operational system.

## 9.2 How the GNN idea actually entered the system

The GNN story came from a disciplined transposition, not cargo culting a paper.

Internal research artifacts show the path:

- a healthcare AI codebase already had anomaly modules and graph-builder patterns
- those patterns were reviewed and decomposed into reusable pieces
- Asolaria's graph runtime was identified as the natural home for a typed temporal heterograph
- the target use case became edge-risk scoring and anomaly detection for the control plane itself

In other words: the GNN idea was moved from a domain-specific healthcare stack into a general operational graph substrate.

### 9.3 The honest current status

This is the important truth:

- the graph runtime is live
- the graph training export path is live
- the sidecar design is live
- the policy kernel is still the root of trust
- there is no evidence that GNN inference currently sits in the critical control path

That is not a weakness. That is the correct design.

If a graph model is going to influence a live system with desktop, phone, cloud, and tool authority, it should begin as:

- ranking support
- anomaly scoring
- explanation support
- secondary signal for review

It should not begin as the thing that directly authorizes dangerous actions.

### 9.4 Why this is still state-of-the-art

The state-of-the-art move here is not "we shoved a GNN into the hot path."

The state-of-the-art move is:

- build the heterograph first
- instrument the system first
- score edges first
- keep the policy kernel authoritative
- make the model a constrained sidecar until the telemetry and governance are mature

That is the grown-up version of GNN integration.

## 10. Skills, Tools, Projects, Memory, and Mistakes

### 10.1 Skills

Asolaria-native skills are declarative, local-first, auditable, and safe-by-default.

Indexed skill ids in the 2026-03-05 snapshot:

- `avatar.npc.business.v1`

- captures.prune.desktop_auto
- captures.prune.desktop_auto.dry_run
- captures.stats
- chrome.profiles_list
- codebase.review_composite_tools_baseline
- codebase.review_external_mcp_cache
- codebase.review_kickoff
- codebase.review_token_efficiency_snapshot
- codex.ref.cloudflare-deploy
- codex.ref.develop-web-game
- codex.ref.doc
- codex.ref.figma
- codex.ref.figma-implement-design
- codex.ref.gh-address-comments
- codex.ref.gh-fix-ci
- codex.ref.google-code-wiki
- codex.ref.imagegen
- codex.ref.jupyter-notebook
- codex.ref.linear
- codex.ref.netlify-deploy
- codex.ref.notion-knowledge-capture
- codex.ref.notion-meeting-intelligence
- codex.ref.notion-research-documentation
- codex.ref.notion-spec-to-implementation
- codex.ref.openai-docs
- codex.ref.pdf
- codex.ref.playwright
- codex.ref.render-deploy
- codex.ref.screenshot
- codex.ref.security-best-practices
- codex.ref.security-ownership-map
- codex.ref.security-threat-model
- codex.ref.sentry
- codex.ref.sora

- `codex.ref.speech`
- `codex.ref.spreadsheet`
- `codex.ref.system.skill-creator`
- `codex.ref.system.skill-installer`
- `codex.ref.transcribe`
- `codex.ref.vercel-deploy`
- `codex.ref.yeet`
- `desktop.capture`
- `desktop.diagnostics`
- `desktop.dual_capture`
- `gemini.enterprise_business_backend_probe`
- `gemini.enterprise_business_open`
- `integrations.snapshot`
- `local.secure_rebuild.lab`
- `mistake.avoidance.hints`
- `phone.browser_history_check`
- `swarm.mode.status`
- `ui.visual_audit`

This matters because the skill layer is not a one-off prompt template folder. It is a declared operational substrate for repeatable work.

## 10.2 Tool actions

Indexed tool actions:

- `tools_status`
- `swarm_mode_status`
- `mistake_avoidance_hints`
- `local_rebuild_plan`
- `codex_skill_reference`
- `google_status`
- `gcp_status`
- `gcp_services_enabled`
- `vertex_status`
- `vertex_budget_status`
- `gemini_api_status`
- `avatar_npc_generate`

- captures_stats
- captures_prune
- captures_mark_important
- desktop_window_list
- desktop_window_active
- desktop_window_focus
- desktop_click
- desktop_double_click
- desktop_scroll
- desktop_type
- desktop_key
- desktop_move
- desktop_capture
- desktop_dual_capture
- ui_visual_audit
- phone_browser_history_check
- chrome_profiles_list
- chrome_open_url
- web_mcp_inspect
- web_mcp_task

## 10.3 Projects

The built-in project store already groups work into persistent operational scopes.

Current tracked projects include:

- Asolaria Core
- AI Healthcare
- Bridge Ops
- Skills Lab
- Task Wave 20260222144718 Updated

Built-in groupings are:

- projects
- tasks
- skills
- runtime

### 10.4 Memory

The memory store is disk-backed, compacted, snapshot-aware, and bounded by turn limits. It is explicitly designed to avoid ballooning into unstructured history.

### 10.5 Mistake capture

This is another standout feature.

Asolaria records mistakes into a ledger and promotes them into a mistake taxonomy. That means the system is not just logging failure. It is turning failure into reusable operational memory.

Representative mistake patterns currently present in the raw store include:

- `swarm_mode_toggle`
- `stale_agent_id`
- `agent_colony_idle_escalation`
- `Duplicate variable naming collision in PowerShell`
- `path_assumption`
- `script_path_mismatch`
- `command_syntax`
- `command_wrapping_error`
- `shell_quoting`
- `regex_quoting`
- `reserved_variable_misuse`
- `auth_assumption`
- `missing_dependency`
- `tool_fallback`
- `command_timeout`
- `formatting_bug`
- later bucketed operational entries such as `sweep_failure`, `capture_on`, `capture_off`, `apply_on`, `apply_off`, and `restore`

The dominant recurring operational issue in one indexed snapshot was not model hallucination. It was `agent_colony_idle_escalation`. That is exactly the kind of systems-level mistake visibility most agent stacks do not capture.

## 11. MCP and Token-Efficient Tooling

This is one of the strongest OpenAI-relevant design points.

Asolaria's MCP posture is not just "supports MCP." It has a token-efficiency and trust-boundary strategy around MCP.

The system-card and benchmark material records a compact-tool strategy built around:

- pointer-based tool schemas
- narrow allowlists
- cached tool discovery
- previous response context reuse

Recorded benchmark values:

- 50 inline function tools: 5382 tokens
- 1 MCP pointer plus allowlist of 5: 82 tokens
- follow-up with previous response context: 42 tokens

That is not a cosmetic improvement. It is the difference between a frontier-model tool turn being bloated versus elegant.

# 12. Multi-Agent, Multi-Machine, and Cross-Device Operation

This is where Asolaria starts to look like a colony rather than an app.

## 12.1 Relay backbone

The relay stack includes:

- local relay server
- room-based messaging
- bearer token auth
- optional persistence
- Cloudflare quick tunnel
- watchdog restart logic
- room policy support
- per-sender token enforcement

## 12.2 Named colony lanes

Current named lanes include:

- `Vector` via `this-codex`
- `Rook` via `other-codex`
- `Forge` via `build-codex`
- `Falcon` via `phone-codex`

These are not symbolic names only. Startup order, identity behavior, and lane semantics are documented.

### 12.3 Windows-to-Mac / friend-machine bootstrap path

The system already includes a "friend Codex bridge pack" and a machine-specific bootstrap system card for bringing up another workstation.

That means Asolaria already has documented patterns for:

- creating a dedicated room per machine
- issuing a profile and environment template without secrets
- syncing a bridge bot to another host
- starting a remote Codex lane with explicit identity
- tailoring bootstrap to Windows, macOS, or Linux host realities

What is directly supported:

- multi-machine relay bootstrap
- cross-machine Codex collaboration
- machine-specific bootstrap planning
- Mac-aware friend-machine onboarding in the bootstrap card

What is not yet proven by a single canonical playbook:

- a one-command full clone-plus-sync fleet replication path

That distinction should be kept explicit.

### 12.4 Phone lane

The phone lane is particularly important because it means Asolaria is not limited to desktop-only agency. It has a dedicated lane for mobile workflow tasks and a documented startup requirement that phone paths be established before the mobile worker is trusted.

## 13. Why OpenAI Should Care

OpenAI should care because Asolaria is already close to the shape of a serious frontier-model operating environment.

It demonstrates:

- Codex as a governed local reasoning lane
- Symphony as a managed long-running execution sidecar
- remote MCP used with compact, policy-aware tool schemas
- desktop and phone actuation under explicit authority modes
- graph-native runtime telemetry instead of opaque agent state
- an actual mistake system instead of generic "logs"

- machine bootstrap patterns for expanding the colony across hosts

In plain terms:

Asolaria shows what users build when they want frontier models to run real workflows, on real machines, with real controls.

This is not "one more chatbot shell."

This is the beginnings of a model-centered operating layer.

# 14. Known Gaps and Honest Limits

A board-facing review should be impressed by the ambition, but it should also trust the honesty.

Current evidence-backed gaps:

- No canonical single-command clone-and-sync fleet replication playbook yet
- No evidence that live GNN inference currently operates in the critical control path
- No single consolidated "external app control matrix" document that unifies phone control, AnyDesk, desktop control, and browser automation under one policy artifact
- Some runtime reports still note lane quality differences, especially around external worker reliability and phone push subscription completeness

These are not fatal weaknesses. They are the normal unfinished edges of a system that is already wider than most agent platforms.

# 15. The Real "Wow"

The most important takeaway is not any single integration.

The real "wow" is that Asolaria already combines:

- model routing
- tool authority
- relay-based colony operation
- desktop and phone actuation
- graph-native runtime instrumentation
- mistake learning
- public-release hygiene
- machine bootstrap patterns

inside one local-first operational system.

That is a much rarer artifact than a model demo.

# 16. Source Base Used for This Card

Primary local sources used for this synthesis:

- README.md
- docs/CANONICAL_SOURCE.md
- docs/STARTUP_CONTRACT_V1.md
- docs/AUTHORITY_MODES.md
- docs/GATEWAY_TOOL_AUTHORITY.md
- docs/PUBLIC_RELEASE_PIPELINE.md
- docs/SYMPHONY_INTEGRATION.md
- docs/AUGMENT_CONTEXT_SIDECAR.md
- src/brainOrchestrator.js
- src/jobQueue.js
- src/projectStore.js
- src/memoryStore.js
- src/mistakePatternStore.js
- src/mistakeLedgerStore.js
- src/graphRuntimeStore.js
- src/connectors/*
- tools/export-graph-runtime-dataset.js
- tools/Friend-Codex-Bootstrap-SystemCard.prompt.md
- tools/New-Friend-Codex-BridgePack.ps1
- tools/Phone-Recovery-Runbook.md
- reports/asolaria-system-card-20260303.md
- reports/asolaria-system-card-20260308-runtime-post-restart.md
- reports/graph-runtime-research-review-20260307.md
- reports/skills-tools-mistakes-index-20260305-015506.json
- codex-bridge/ASOLARIA_COLONY_STARTUP.md
- codex-bridge/AGENT_PROFILE_SPEC.md
- codex-bridge/AIRLOCK_BOTS.md
- codex-bridge/watchdog.ps1